ZARTEX SYSTEMS ENGINEERING INITIATIVE

Building Production-Grade Distributed Systems as a Team

5 Members · Beginner Team · Backend & Distributed Systems · 6-7 Months

---

A. OVERVIEW

Who We Are

We are five students with no industry experience and a shared goal: build a portfolio that gets us shortlisted at serious engineering companies. We are not pretending to be senior engineers. We are committing to work like them - writing real code, reviewing each other's work, testing under load, and shipping things that actually run.

This document is the single source of truth for our initiative. It covers what we are building, why we are building it, how we plan to do it together, and what every member is responsible for. If you are reading this, you have agreed to hold yourself to everything in it.

The Problem with Most Student Portfolios

Most student projects look identical to a recruiter: a full-stack CRUD app, a machine learning notebook, or a clone of an existing product. None of them demonstrate systems thinking, and none of them hold up under a technical interview question like 'walk me through how your system handles failure at scale.'

We are building systems that answer that question directly - because we will have actually built them.

What We Are Building

- Distributed Rate Limiter as a Service - traffic control with sub-millisecond latency
- Distributed Task Queue Engine - async job processing with retry logic and durability
- Append-Only Event Store with Replay Engine - durable event sourcing infrastructure

Target Outcome - For Every Member

- A GitHub profile that shows real engineering depth, not course certificates
- At least one live deployed system with benchmark numbers we can cite in interviews
- Deep, working knowledge of distributed systems concepts - not just definitions
- The ability to whiteboard and defend every architectural decision we made
- Shortlist-level positioning for backend, SDE, and infrastructure roles at FAANG and strong startups

---

B. PROJECT BREAKDOWN

We build in this exact order. Each project teaches the mental models the next one requires. Skipping ahead or parallelizing will result in restarts.

Project 1 - Distributed Rate Limiter as a Service *Token bucket · Sliding window · Redis atomicity · gRPC · Sub-5ms p99*	
What It Is	A standalone service that controls how many requests a client can make within a time window. It coordinates state across multiple nodes using Redis and supports multiple rate limiting strategies, switchable per client via a config API. Think of this as the system that sits in front of every API at Stripe or Google - preventing abuse without adding latency.
Why This First	It is the most contained of the three projects. You learn Go (or Java), Redis, Docker, and distributed state management in a system with a clear, testable contract. The complexity is real - Redis Lua scripting, clock skew, atomic operations across nodes - but the scope is small enough that a beginner team can ship it cleanly.
Why It Signals Well	Rate limiting is a canonical system design interview question. Every senior engineer who looks at this project will immediately understand what it demonstrates. Building it from scratch - not importing a library - proves you understand the problem, not just the API.
Top 1% Features	- Token bucket, sliding window, and fixed window algorithms - switchable per client - Redis Lua scripts for atomic read-modify-write across distributed nodes - gRPC interface with REST fallback - Per-user, per-IP, and per-endpoint rule enforcement via config API - Sub-5ms p99 response latency under load - benchmarked and documented - Correctness tests: no over-counting or under-counting across nodes under concurrent load
Stack & Proof	**Stack:** Stack: Go · Redis · gRPC · Docker · Docker Compose · k6 - Load test results at 1K, 5K, and 10K requests/second with latency breakdowns - Distributed correctness test suite - proves no race conditions across nodes - Architecture diagram, README, and a written section on scaling beyond this design

Project 2 - Distributed Task Queue Engine *Async job processing · Retry logic · Concurrency · Dead-letter queue · Durability*	
What It Is	A Redis and PostgreSQL-backed job queue with configurable worker pools, retry logic with exponential backoff, dead-letter queues for failed jobs, and a scheduling layer for delayed execution. This is how email delivery, payment retries, image processing, and notification systems work behind the scenes at every serious company.
Why This Second	You already understand Redis and Docker from Project 1. Now you add PostgreSQL, concurrent worker management, and failure recovery - the complexity of the system goes up significantly, but the foundational knowledge is already there. This is intentional sequencing.
Why It Signals Well	Async processing is not optional at scale - it is a fundamental pattern. Building a task queue from scratch demonstrates understanding of at-least-once delivery, idempotency, worker coordination, and graceful degradation under failure. These are topics that come up in every backend systems interview.
Top 1% Features	- Priority queues with delayed and scheduled job support - Configurable worker thread pools with concurrency limits per queue - Exponential backoff retry logic with per-job retry budgets - Dead-letter queue with failure reason logging and manual retry API - Admin dashboard: real-time job status, throughput, failure rate, and queue depth - Prometheus metrics and Grafana dashboard for observability
Stack & Proof	**Stack:** Stack: Go · PostgreSQL · Redis · Docker · Prometheus · Grafana · k6 - Load test results showing 5,000 to 10,000 jobs per minute sustained throughput - Worker crash recovery benchmark - system continues processing after node failure - At-least-once delivery correctness test under simulated failure conditions

Project 3 - Append-Only Event Store with Replay Engine *Event sourcing · Consumer groups · Ordering guarantees · Compaction · WAL-inspired design*	
What It Is	A lightweight event sourcing engine with an append-only write path, sequence-numbered and checksummed events, consumer group subscriptions with offset tracking, and a full state replay engine that can reconstruct the state of any object from any point in the log. A mini Kafka and event store hybrid - built from first principles using PostgreSQL as the durable foundation.
Why This Last	This project demands everything - durable storage design, strict ordering guarantees, consumer group coordination, compaction logic, and a correctness standard that does not allow for approximation. You will not be ready for this without the first two projects. That is not a guess - it is why this order exists.
Why It Signals Well	Event-driven architecture is foundational at companies like LinkedIn, Uber, and Shopify. Building an event store from first principles signals understanding of write-ahead log design, ordering guarantees under concurrent writes, and the CAP tradeoffs that most candidates can recite from a blog post but cannot implement.
Top 1% Features	- Append-only write path with monotonic sequence numbers and SHA checksums - Consumer groups with independent per-group offset tracking - at-least-once delivery - Full state replay from any sequence offset for any aggregate - Log compaction with configurable retention policy to manage storage growth - HTTP and gRPC read/write API with Protobuf schemas - Written comparison document: our design vs Apache Kafka - tradeoffs, not just differences
Stack & Proof	**Stack:** Stack: Go · PostgreSQL · Docker · gRPC · Protobuf · k6 - Write throughput benchmark targeting 20,000 events per second - Replay correctness test suite - state after replay matches state at write time - Failure simulation tests: partial writes, mid-compaction crashes, consumer lag recovery

C. EXECUTION PLAN

Realistic Duration

The original estimate of 22-26 weeks assumed someone already comfortable with Go, Docker, and basic backend patterns. Our team is starting from zero on most of this.

This is not a failure to go slower. It is accurate planning. A slower timeline that ships three real systems is worth more than a faster timeline that ships three broken ones.

No parallelization across projects. Each project builds the foundation the next one requires.

Full Timeline Breakdown

Phase	Duration	What Gets Built
Learning Runway	Weeks 1-2	Go fundamentals, Docker, Redis basics, Git workflow, how to read technical documentation, first PR submitted
Project 1: Rate Limiter	Weeks 3-9	Core algorithms → Redis integration → Lua scripting → gRPC layer → Load testing → Benchmarks → README and diagram
Project 2: Task Queue	Weeks 10-16	Queue + persistence → Worker pools → Retry logic → Dead-letter queue → Observability → Benchmarks → README and diagram
Project 3: Event Store	Weeks 17-23	Append-only writes → Consumer groups → Replay engine → Compaction → Benchmarks → Kafka comparison doc
Polish and Deploy	Weeks 24-26	Live deployment of one system, portfolio cleanup, interview prep, mock system design walkthroughs

Where Beginner Teams Lose the Most Time

- Docker and networking setup - do not touch project code until the team can run a container confidently. Budget a full week just for this.
- Debugging concurrency bugs - these are silent, intermittent, and take days without a trained eye. Pair-debug everything in this category.
- Building too much before testing anything - write the simplest version, test it, then extend. Do not code for three days and test once.
- Scope creep - adding features before the core is solid. Freeze the feature list after week one of each project.
- Documentation left to the end - write the README as you build. It takes twice as long if done in one sitting at the end.

Weekly Discipline

Every week has three things: a defined deliverable, a check-in, and a written status update in the team channel. If a week ends without a deliverable, the next week's plan adjusts to account for the gap - it does not carry over silently.

Daily Hours	What This Realistically Achieves
1 hour / day	Below the minimum. Progress will stall on hard problems. Not recommended.
1.5 hours / day	The minimum viable commitment. Achievable with discipline and protected blocks.
2 hours / day	On track. Leaves buffer for debugging and the inevitable setup days.
2.5+ hours / day	Ahead of schedule. Use the surplus for depth, not shortcuts.

D. TEAM STRUCTURE

How We Are Organized

We have five members. Roles are assigned per project and rotate - no one is permanently the infrastructure person or the documentation person. Every member will write core logic, review code, run tests, and write documentation across the initiative. The point is to graduate as full engineers, not specialists in one narrow slice.

One exception: the Infrastructure role for Project 1 should be held by whoever is most willing to go deep on Docker and tooling. They will set up the patterns the whole team uses for all three projects. After that, the role rotates.

Role Definitions

Role	What They Own	Notes
Backend Lead	Core system logic, data structures, API layer, performance-critical code. Makes final call on architecture within the project.	Rotates each project
Backend Support	Pairs with the lead on implementation. Owns unit test coverage and internal code review. Catches what the lead misses.	Pairs with lead and Rotates each project
Infrastructure & DevOps	Docker Compose, CI/CD pipeline, Prometheus, Grafana, cloud deployment. Owns the 'it runs in one command' guarantee.	Stays consistent - build this skill deep
Testing & Benchmarks	k6 load test setup, correctness test suites, failure scenario simulation. Owns every benchmark number we publish.	Rotates each project
Docs & Architecture	README, architecture diagrams (Excalidraw), ADRs, the 'what we'd do at scale' section. Owns the story of what we built and why.	Stays consistent - build this skill deep

Role Rotation Plan

Member	Project 1 Role

The role rotation table should be filled in by the team at kickoff based on individual interests and strengths. The constraint: no member can hold the same role twice across the three projects.

Collaboration Model

- All code goes through pull requests. No direct commits to main - for any reason, ever.
- Every PR requires at least one teammate review before merge. The reviewer must leave a comment - not just an approval click.
- Weekly 30-minute sync: what shipped last week, what is blocked, what ships this week. Async updates in the team channel for days when sync is not possible.
- Architecture decisions are documented as ADRs (Architecture Decision Records) - short Markdown files in the repo that record what we decided and why. Future us will thank current us.
- Blocked for more than 2 hours? You post in the group. Sitting on a blocker silently is the fastest way to fall behind.

E. NON-NEGOTIABLE STANDARDS

These are the minimum requirements for a project to be considered complete. Not complete enough - complete. Every item below applies to every project. There are no partial credits.

Code and Architecture

- Clean, readable code with consistent naming conventions - agreed on in week one and enforced in every PR
- No dead code, commented-out blocks, or TODO comments on main
- All major components have at least basic unit test coverage
- Every non-obvious architectural decision is documented - in code comments or ADRs

Performance and Correctness

- Every project has a load test - written with k6, with results documented in the README
- Benchmark numbers are specific: latency at p50, p95, p99; throughput at defined concurrency levels
- Correctness tests exist for distributed edge cases - concurrent writes, worker failure, network partition simulation
- Failure modes are documented: what breaks, how it recovers, what the known limits are

Developer Experience

- One-command startup via Docker Compose - if it takes more than one command, it is not done
- README explains what the system does, how to run it, how to run tests, and how to run load tests
- Architecture diagram present and up to date - updated when the design changes, not only at project end
- A written section titled 'What We Would Do at 10x Scale' in every README

Deployment

- At least one of the three projects is deployed to a live cloud server (Railway, Render, or a cheap VPS)
- The deployed system has a public URL that can be shared in interviews
- Basic monitoring is live on the deployed system - not optional

F. EXPECTATIONS

Time Commitment

The minimum is 1.5 focused hours per day, five days per week. That is roughly seven to eight hours per week per person. If that is not realistic for your schedule right now - because of exams, internships, or other commitments - the right move is to delay joining, not to join and deliver half the work.

Distributed systems bugs do not yield to fragmented attention. You need sustained focus to make progress. Twenty-minute sessions between classes will not move the needle.

What Being a Beginner Means on This Team

It means we do not know everything before we start - and we do not pretend otherwise. It does not mean we cut corners, skip documentation, or ship untested code because 'we are still learning.'

- You are not expected to know the answer before googling it.
- You are expected to read the documentation, not just the first Stack Overflow answer.
- You are expected to understand the code you write - not just copy it.
- You are expected to explain your decisions in a PR description.
- You are expected to ask for help before a blocker turns into a missed week.

Ownership

You own your role end to end for the project you are assigned to. That means design, implementation, testing, documentation, and the debugging at 11pm before the week's deliverable is due. You do not hand off a half-finished module. You do not wait to be assigned a subtask. You figure out what needs doing and you do it.

The Quality Bar

The question for every piece of work - every PR, every README section, every benchmark - is this: would a senior backend engineer at a top company look at this and take it seriously?

If the answer is no, it goes back for rework. There is no other path to merge.

G. WHY JOIN THIS TEAM

What You Build

- Three distributed systems that simulate real production infrastructure - not tutorial clones
- Benchmark numbers and failure recovery demos you can walk through in any technical interview
- A live deployed system with a URL - something almost no student portfolio has
- A GitHub history that shows consistent, reviewed, production-standard engineering work

What You Learn

- Go - one of the most in-demand languages for backend and infrastructure engineering
- Redis, PostgreSQL, Docker, gRPC, Prometheus, Grafana - the standard production backend stack
- Distributed systems fundamentals: concurrency, consistency, retries, ordering, fault tolerance
- Real engineering collaboration: pull requests, code review, architecture decisions, debugging together
- How to read technical documentation and implement from a spec - not just follow a tutorial

What This Does for Your Placement

The gap between a student who gets filtered out and one who gets a phone screen is not GPA. It is not LeetCode count. It is evidence - specific, verifiable, technical evidence that you have built something real, understood why it works, and can defend the decisions you made.

Every project in this initiative is designed to produce exactly that evidence. When an interviewer asks 'tell me about a system you built and the tradeoffs you faced,' you will have a real answer - with numbers, diagrams, and deployment proof behind it.

That is the difference between a resume that gets read and one that gets skipped.

H. COMPENSATION & DISCIPLINE MODEL (NON-NEGOTIABLE)

This initiative operates with the seriousness of a paid internship. The structure below is designed to enforce consistency, accountability, and professional discipline - not comfort.

Compensation Structure

- Each member is allocated a total value of **₹5000 (cash + stock equivalent)** over the duration of the initiative
- This is **earned, not guaranteed** - payouts are strictly performance-based
- Final distribution is proportional to consistency, contribution, and adherence to rules

Work Commitment

- **5 working days per week - mandatory**
- No exceptions for exams, internal assessments, or personal schedules
- If you cannot sustain this, you should not be in this team

Attendance Penalty Model

- Missing a scheduled workday results in **direct proportional deduction** from the total ₹5000 pool
- Example: Missed days reduce your final payout percentage - no negotiations, no recovery claims

Consistency Penalties (Micro-Discipline Layer)

- **LeetCode streak break → ₹5 deduction per occurrence**
- **GitHub commit streak break → ₹20 deduction per occurrence**

These are not symbolic penalties - they exist to enforce **daily engineering rhythm**

Why This Exists

- Real engineering teams don't reward intent - they reward output and consistency
- Discipline compounds; inconsistency kills momentum
- This system ensures everyone operates at a **baseline professional standard**, not a student mindset

Exit Clause

- If you are unable or unwilling to comply:
    - You may leave early
    - Compensation will be adjusted strictly based on contribution till exit
    - No partial entitlement claims

We are beginners. The work will not be. That is the point.